Beyond File Scripting: A Contract-Guarded Dual-Surface CRDT and Token-Patched OOXML Engine for Human-AI Scientific Co-Authoring

Hui Zhao, Heurion Research Team
Correspondence: `research@heurion.ai`

November 2026

Abstract

Agent-Computer Interfaces (ACIs) have transformed autonomous software engineering by replacing raw shell manipulation with structured, lint-guarded code editors. However, scientific and clinical manuscript co-authoring still relies largely on free-form file scripting (e.g., rewriting Office Open XML `.docx` archives via `python-docx` or Markdown round-trips). In collaborative workflows, this paradigm suffers from three structural failure modes: (1) editing even a single phrase in a paragraph strips intra-paragraph publisher XML constructs such as EndNote complex citation fields (`ADDIN EN.CITE`), dynamic numbering fields, footnotes, inline equations, co-author tracked changes, and threaded comment anchors; (2) document-level optimistic version checks (`ETag`/`base_rev`) fail in multi-turn sessions when an agent obtains a fresh revision on one paragraph and blindly overwrites an unread clinician edit or deletes a clinician-corrected statistical figure in another; and (3) file snapshots cannot selectively undo an earlier AI turn once a human co-author edits a different clause inside the same paragraph.

We present the **Heurion Agent-Document Interface (ADI)**, a collaborative document engine built on three mechanisms. First, a *Dual-Surface CRDT Core* (ProseMirror + Yjs) projects a live rich-text editor for human authors and a compact Annotated Markdown view for AI agents with persistent block IDs (`{#id}`), 2D logical table coordinates, atomic citations (`[@c:id]`), opaque Word field/footnote atoms (`[@w:id]`), and comment anchors (`{#c:id}`). Second, *Multi-Granularity Pre-Write Contracts* pair side-effect-free pre-flight validation (`doc_validate`) with per-block read-horizon tracking (`noteSeen`), clinician-delta protection, lock-free intra-paragraph phrase commuting (`replace_text`), and a two-tier *Selective AI Turn Undo* (`revertTurn`) spanning in-memory CRDT transaction origins and cold-restart SQLite fallback. Third, an *Intra-Paragraph Token-Patched OOXML Engine* (`docx-patch`) writes untouched blocks byte-for-byte from a raw slice cache (`node_src`) while surgically patching only altered character runs inside modified paragraphs to keep surrounding Word fields, equations, bookmarks, tracked revisions, and threaded comments (`w15:paraIdParent`) intact. Evaluated across 19 open-access 2025 PubMed Central full-length clinical manuscripts (150,152 words, 2,719 AST blocks across 7 study designs), Heurion ADI preserves 100.0% byte identity across all 2,553 untouched blocks and 100.0% (114/114) of publisher XML constructs inside edited blocks—compared to 0.0% for Markdown re-serialization and 16.7% (19/114) for `<w:pPr>`-preserving XML paragraph scripts—at median latencies of 2.9 ms for validation, 5.3 ms for selective undo, and 15.5 ms for patched `.docx` export, while achieving 100.0% accuracy across all 8 deterministic co-editing and out-of-order undo scenarios (152/152 trials). In a live multi-turn agent evaluation (171 episodes driven by `deepseek-v4-flash` at $T = 0$), Heurion ADI achieves 100.0% (57/57) end-to-end task and OOXML fidelity—versus 33.3% (19/57) for direct OOXML LLM editing (with 4.3× fewer output tokens and 3.2× lower latency on clinical table edits) and 0.0% (0/57) for document-`ETag` MCP—autonomously self-repairing 100.0% (14/14) of mid-session clinician edit conflicts within the same turn.

Keywords: Agent-Document Interface, Human-AI Co-Authoring, Rich-Text CRDT, Office Open XML, Selective Undo, Clinical Research Informatics

1 Introduction

Recent advances in coding agents such as SWE-agent [1] and OpenHands [2] demonstrated a foundational systems principle: autonomous large language model (LLM) agents are substantially more reliable when equipped with a purpose-built *Agent-Computer Interface (ACI)*—offering compact structural views, atomic edit primitives, and immediate syntactic guardrails—rather than raw shell scripts. Yet when LLM agents are deployed to assist investigators in authoring peer-reviewed clinical and scientific manuscripts [3, 4, 5], most existing workflows revert to treating Office Open XML (`.docx`) [6] manuscripts as static disk files. Agents either convert manuscripts through lossy Markdown pipelines or generate ad-hoc scripts (e.g., `python-docx`) to mutate XML paragraphs on disk.

Deploying script-based editing in real multi-author clinical studies exposes three fundamental systems bottlenecks:

1. **Intra-Paragraph Publisher XML Destruction:** Clinical manuscripts circulate among co-authors and journal production systems carrying rich OOXML structures that have no plain-Markdown equivalent: multi-run EndNote or Zotero citation fields (`ADDIN EN.CITE`), dynamic caption counters (`SEQ Table`), footnotes, inline Office Math (`<m:oMath>`), cross-reference bookmarks, content controls (`<w:sdt>`), co-author tracked changes (`<w:ins>`/`<w:del>`), and threaded Word review comments (`commentsExtended.xml`). While a careful XML script can leave *untouched* paragraphs intact, rewriting the text of a modified paragraph (`paragraph.text = ...`) flattens its internal XML runs, silently destroying embedded citation fields, footnotes, equations, and comment range anchors.
2. **Multi-Turn Blind Overwrites Beyond Document-Level `ETags`:** In interactive co-authoring, an agent reads a manuscript at revision r_0, a clinician manually corrects a hazard ratio (HR) in paragraph p_1 (r_1), and the agent subsequently edits paragraph p_2 (r_2). Because the agent now holds the latest document revision r_2, standard document-level optimistic concurrency (`base_rev == current_rev`) allows the agent to overwrite p_1 from stale context memory without ever having read the clinician's edit. Even after re-reading p_1, an LLM asked to "make the paragraph more concise" frequently drops clinician-corrected numbers or newly added caveat sentences, or injects unregistered bibliographic identifiers [7].
3. **Non-Commutative File Snapshots vs. Intra-Paragraph Selective Undo:** Suppose an AI turn revises two paragraphs (p_1, p_2), a clinician immediately edits a second clause inside p_2, and a subsequent AI turn updates p_3. If the clinician then decides to revert the first AI turn, linear undo (`Ctrl+Z`) or file-level snapshot restore wipes out both the clinician's manual edit inside p_2 and the later AI turn in p_3.

To resolve these bottlenecks without relying on fragile prompt engineering, we design and evaluate the **Heurion Agent-Document Interface (ADI)**, a contract-guarded collaborative document engine that decouples collaborative editing state from OOXML delivery archives:

- **Dual-Surface Synchronized CRDT Core (Section 2):** A server-authoritative ProseMirror + Yjs CRDT state [8, 9, 10] projects a real-time rich-text WebSocket editor for human authors and an Annotated Markdown surface for AI agents featuring lifecycle-stable block IDs (`{#id}`), 2D logical table grids, registered citation atoms (`[@c:id]`), opaque Word inline atoms (`[@w:id]`), and comment anchors (`{#c:id}`).
- **Horizon-Tracked Co-Editing Contracts & Two-Tier Selective Undo (Section 3):** Pre-write validators enforce per-block AI read horizons (`noteSeen`), post-read clinician-

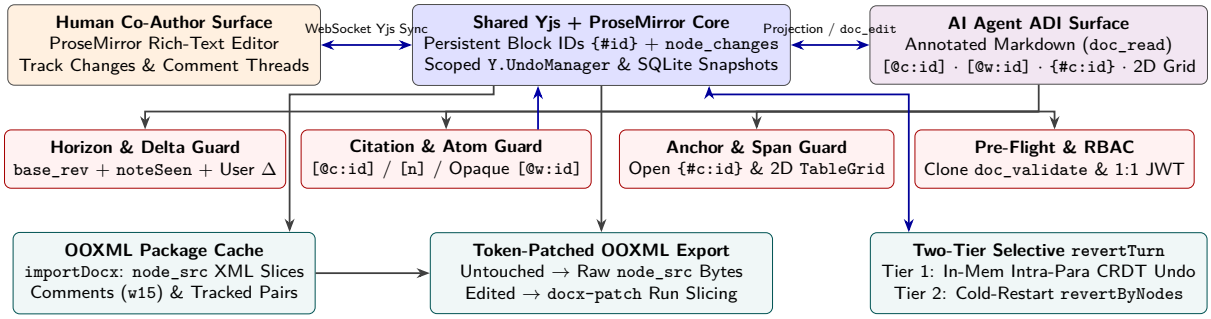


Figure 1: Architecture of the **Heurion Agent-Document Interface (ADI)**. Human co-authors and AI agents operate on a single Yjs + ProseMirror CRDT document through synchronized projections. Every AI edit batch passes through deterministic pre-write contracts before mutating CRDT state, supports intra-paragraph out-of-order turn undo, and exports back to publisher `.docx` via raw block caching (`node_src`) and intra-paragraph token-level XML patching (`docx-patch`).

delta preservation, comment-anchor and native-citation protection, side-effect-free pre-flight probes (`doc_validate`), and a two-tier `revertTurn` combining turn-scoped CRDT `Y.UndoManager` origins with persistent SQLite block fallback (`revertByNodes`) across restarts.

- **Intra-Paragraph Token-Patched OOXML Engine (Section 4):** On `.docx` export, untouched blocks write back byte-for-byte from cached slices (`node_src`), while modified paragraphs undergo token-aligned XML diffing (`docx-patch`) that updates only changed `<w:t>` character slices while preserving surrounding EndNote fields, footnotes, equations, bookmarks, co-author tracked changes, and threaded Word comments.

2 Dual-Surface CRDT Core and Agent-Document Interface

Office Open XML (`.docx`) archives interleave content with layout caches, RSIDs, namespace prefixes, and split character runs, making raw XML both token-inefficient and error-prone for LLMs. Conversely, converting a manuscript to plain Markdown discards inline structural metadata. As shown in Figure 1, Heurion ADI resolves this tension by maintaining a single server-authoritative Yjs CRDT document (`Y.Doc`) [9] backed by SQLite snapshots and an append-only block change log (`node_changes`), projecting two synchronized views:

Lifecycle-Stable Block Addressing (`{#id}`) and 2D Logical Table Grids. Every structural node (heading, paragraph, list item, table, cell paragraph, or figure) carries a persistent short identifier `{#id}`. When a browser user splits a paragraph or pastes blocks, a server transaction automatically assigns fresh unique IDs to deduplicate collisions. Multi-column clinical tables containing horizontal (`colspan`) and vertical (`rowspan`) header merges are projected through a unified 2D logical coordinate mapper (`readGrid`): merged continuations render explicitly as `<` and `^` in the agent's Markdown view, ensuring that the (r, c) coordinates read by the agent map identically to `table_set_cells`, `table_merge_cells`, and `table_split_cell`.

Opaque Word Inline Atoms (`[@w:id]`) vs. Registered Citations (`[@c:id]`). To let agents read and edit paragraphs containing complex Word constructs without exposing raw XML, `importDocx` lifts non-textual inline OOXML structures—footnotes (`<w:footnoteReference>`), simple fields (`<w:fldSimple>`, such as `SEQ Table`), and multi-run complex fields (`w:fldChar begin...end`, such as EndNote `ADDIN EN.CITE`)—into atomic inline nodes rendered

in Annotated Markdown as `[@w:w1]` alongside a read-only legend at the bottom of the agent's viewport:

```
{#p8f2} Supplementary Table [@w:w1] summarizes baseline adjudication[@w:w2] for the cohort.
---
[@w:w1] Field code (displayed as "1")
[@w:w2] Footnote: Adjudicated per independent clinical endpoint committee charter.
```

Platform-managed citations are similarly represented as atomic nodes `[@c:cite_id]` registered via Crossref, PubMed, or OpenAlex, whereas open review comment spans appear inline with thread IDs and quoted snippets.

3 Multi-Granularity Co-Editing Contracts and Selective Undo

All AI mutations execute through atomic batches (`doc_edit`) supporting both direct application (`mode: "apply"`) and native tracked-changes proposals (`mode: "suggest"`). Before any Yjs transaction is committed, `OpService.simulate` validates the entire batch against an ephemeral ProseMirror state; if any operation fails, the batch aborts atomically and returns a structured `OpError` containing an error code, recovery hint, and current block text. Agents can also invoke this exact pipeline without side effects via `doc_validate` (or `dry_run: true`).

3.1 Per-Block Read Horizons (`noteSeen`) and Clinician-Delta Protection

Document-level optimistic locking (`base_rev`) only detects edits that occur *after* the latest revision number held by the agent. In multi-turn sessions, an agent often edits section B (receiving fresh revision r_2) while holding stale context for section A, which a clinician edited earlier at $r_1 < r_2$. Heurion ADI closes this gap at three granularities:

1. **Per-Block Read Horizon (`noteSeen`):** Whenever `doc_read` returns block b at revision r, the server records $\text{seen}(b) \leftarrow r$. Even if an agent supplies the latest document `base_rev`, any whole-block write (`replace_block`, `delete`) on b is rejected with `conflict_user_edited` if a human modified b at any revision $r' > \text{seen}(b)$, returning the clinician's updated text in `extra.current`.
2. **Post-Read Clinician-Delta Guard:** Clinicians frequently correct statistical figures (e.g., changing `HR 1.82` to `HR 1.12`) or append methodological caveats, then ask the AI to polish prose. Even *after* the AI has re-read the block ($r' \leq \text{seen}(b)$), the validator extracts user-authored numeric tokens and added sentences from `node_changes` and blocks any AI `replace_block` or `replace_text` that silently drops them (`conflict_user_edited`).
3. **Lock-Free Intra-Paragraph Commuting (`replace_text`):** For surgical phrase edits, the literal `find` string acts as a fine-grained precondition: if a clinician edited clause 1 of paragraph b after `base_rev`, an AI `replace_text` targeting clause 2 of b still matches uniquely and commits cleanly, slicing only the target character offsets in ProseMirror and Yjs while preserving surrounding bold/italic marks, `[@c:id]` citations, and comment highlights [10]. Ambiguous multi-matches require an explicit `occurrence` index (`ambiguous_match`).

3.2 Citation, Opaque Field, Comment-Anchor, and Permission Guards

Four domain invariants prevent silent corruption of scholarly metadata: (i) *Pre-Write Citation Guard:* scans outgoing text for bare DOIs, bare PMIDs, or hand-typed bibliography lines (`citation_not_registered`), and blocks unknown `[@c:id]` tokens (`citation_unknown`); (ii) *Native Reference & Field Protection:* forbids AI edits from hijacking pre-existing numbered citations such as `[1]` into platform citations (`existing_citation`, which would corrupt external reference lists) or fabricating non-existent Word field markers such as `[@w:w99]`; (iii)

Comment-Anchor Preservation: blocks edits that erase the text anchored by an open review comment unless the batch explicitly passes `ack_comments` or executes as the designated reply turn (`anchor_has_open_comments`), while `merge` automatically migrates comment anchors and footnotes across joined paragraphs; and (iv) *In-Process Permission Symmetry:* every MCP tool executes in-process through the Hono HTTP router under a short-lived user JWT tagged with `X-Heurion-Via: ai` (audited in CI by `parity.test.ts`), gating destructive endpoints behind interactive UI confirmation cards.

3.3 Two-Tier Per-Turn Selective AI Undo (`revertTurn`)

To allow clinicians to undo a specific prior AI turn T_k out of order without losing subsequent human or AI edits, `Documents.revertTurn` combines two complementary tiers:

- **Tier 1 (In-Memory Intra-Paragraph CRDT Undo):** Every AI commit tags its Yjs transaction with a turn-specific `ServerOrigin` (`turnId:` T_k) tracked by a dedicated `Y.UndoManager` scope. Invoking `revertTurn(`T_k`)` inverts only the Yjs item insertions and deletions originated by T_k, preserving concurrent human edits even inside the exact same paragraph as well as later AI turns $T_{k'>k}$.
- **Tier 2 (Persistent Cold-Restart Block Fallback):** Before an AI turn's first mutation, the engine persists the pre-turn Yjs state vector and per-block change manifest in SQLite. If the server restarts before `revertTurn(`T_k`)` is invoked (evicting in-memory `Y.UndoManager` instances), `revertByNodes` reconstructs the pre-turn baseline tree, removes blocks added by T_k, restores modified or deleted blocks to their pre-turn state, and safely skips any block modified by a human after T_k (*user-priority skip*).

4 Intra-Paragraph Token-Patched OOXML Round-Trip Engine

Caching raw XML slices (`node_src`) per imported block guarantees byte-identical export for *untouched* blocks, but falls back to full re-serialization as soon as an agent fixes a typo inside a paragraph. To preserve publisher XML inside *modified* blocks, Heurion ADI introduces a token-level XML patcher (`docx-patch.ts`).

Token-Aligned Run Slicing (`docx-patch`). During export of a modified paragraph b, `docx-patch` parses its original raw XML `node_src(`b`)` into a structural run tree (`RNode`) while simultaneously flattening both the import-time baseline ProseMirror node b_0 and the current ProseMirror node b_t into character-and-atom token sequences (`Tok`, carrying text characters, mark signatures, `[@w:id]` atoms, and `[@c:id]` citations). After verifying 1-to-1 token alignment between `node_src(`b`)` and b_0, a minimal sequence diff between b_0 and b_t patches `node_src(`b`)` directly:

1. Unchanged character runs keep their exact original `<w:r>` XML, preserving unmodeled run properties (custom fonts, theme colors, kerning, proofing flags, RSIDs).
2. Newly inserted characters inherit the non-modeled `<w:rPr>` attributes of the adjacent run (mirroring interactive typing in Microsoft Word), splitting into a sibling run only when explicit formatting marks (e.g., bold/italic) change.
3. Non-textual XML siblings and containers—including complex fields (`w:fldChar begin...end` for EndNote `ADDIN EN.CITE`), `<w:fldSimple>` counters, `<w:footnoteReference>`, bookmarks (`<w:bookmarkStart/End>`), permission ranges (`<w:permStart/End>`), symbols (`<w:sym>`), inline OMML equations (`<m:oMath>`), outer content controls (`<w:sdt>`), and nested or tracked table rows (`<w:trPr><w:ins>`)—remain frozen in their exact original XML positions.

Bidirectional Word Tracked Changes and Threaded Comments. Co-author Word tracked changes (<w:del>/<w:ins>) import as paired ProseMirror blocks (suggest: "delete" | "insert", linked via suggest_of) displaying author attribution in the agent's view. Pre-write guards block AI edits that collide with a co-author's tracked span while allowing edits elsewhere in the same paragraph, which docx-patch writes back into the original XML without altering the co-author's w:author revision tags. Conversely, AI edits made under mode: "suggest" export as native Word <w:del>/<w:ins> pairs while stripping w14:paraId from the deleted twin so every paragraph in word/document.xml retains a unique w14:paraId. Similarly, Word comments and replies (comments.xml + commentsExtended.xml) import as anchored threads; AI thread replies export as child <w:comment> entries linked to the root comment's paragraph ID via w15:paraIdParent, with <w:commentRangeStart/End> paired cleanly even when comments and suggest revisions coexist in the same paragraph.

5 Empirical Evaluation

We evaluate Heurion ADI across two clean-room benchmark suites (paper/benchmarks/run-clean-adi-eval.ts and paper/benchmarks/run-online-adi-llm-eval.ts) over all **19 open-access 2025 PubMed Central (PMC) full-length clinical manuscripts** in our evaluation corpus, spanning 7 study designs (randomized controlled trials, retrospective cohorts, case-control, diagnostic accuracy, cross-sectional, meta-analyses, and translational studies). Across the 19 full-text manuscripts, the corpus totals **150,152 words** (mean 7,903 words/manuscript) and **2,719 AST blocks** (mean 143 blocks/manuscript; 18 of 19 manuscripts contain multi-column clinical tables). All baselines execute as real code pipelines or live tool-calling LLM loops over the exact same full-length .docx archives without hand-estimated numbers or prompt handicapping.

5.1 Experiment 1: Publisher OOXML Round-Trip Fidelity and Latency

For each of the 19 full-length PMC manuscripts, we construct a valid OOXML .docx archive with unique w14:paraId attributes on every paragraph plus publisher collaboration blocks containing: (1) an EndNote ADDIN EN.CITE complex field, (2) a SEQ Table dynamic field and <w:footnoteReference> (footnotes.xml), (3) a bookmark range, <w:sym>, highlight run, and inline <m:oMath> formula, (4) a <w:sdt> structured content control, (5) co-author <w:del>/<w:ins> tracked changes and a tracked nested table row (<w:trPr><w:ins>), and (6) a threaded co-author comment pair (comments.xml + commentsExtended.xml w15:paraIdParent). Each pipeline applies the exact same revision batch (editing an organic PMC prose paragraph plus text inside each of the 6 publisher constructs, alongside a tracked revision). We compare three executable pipelines:

- **Markdown Re-Serialize:** Round-trips the manuscript through Markdown text before re-exporting .docx (representative of Pandoc/Markdown document agents).
- **XML <w:p> Run-Rewrite Script:** Unzips .docx, leaves all untouched blocks 100% verbatim, and updates modified <w:p> elements while preserving their opening <w:p w14:paraId="..."> attributes and <w:pPr> paragraph properties (faithful equivalent of python-docx paragraph.text = ... scripts).
- **Heurion ADI (Ours):** Imports into the Dual-Surface Yjs CRDT core with node_src caching, applies guarded OpService.edit batches (apply + suggest) and thread replies, and exports via docx-patch.

Table 1 highlights why *intra-paragraph* token patching is essential. Markdown re-serialization rewrites the entire document body (0/2,553 untouched block identity) and

Table 1: OOXML round-trip fidelity across $N = 19$ full-length 2025 PMC clinical manuscripts (150,152 words; 2,553 untouched blocks; 114 edited publisher-construct blocks across 6 categories).

OOXML Fidelity Dimension	**Markdown Re-Serialize**	**XML `<w:p>` Script (`pPr`-Preserving)**	**Heurion ADI (`node_src` + `docx-patch`)**
Untouched Block Byte-Identity (2,553 blocks)	0.0% (0/2,553)	**100.0**% (2,553/2,553)	**100.0**% (**2,553/2,553**)
Intra-Block Publisher Construct Survival Inside Modified Paragraphs / Tables ($N = 19$ per category):			
EndNote Complex Field (`ADDIN EN.CITE`)	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19/19**)
`SEQ` Caption Field + Footnote Reference	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19/19**)
Bookmark + `<w:sym>` + Highlight + `<m:oMath>`	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19/19**)
Outer Content Control Wrapper (`<w:sdt>`)	0.0% (0/19)	**100.0**% (19/19)	**100.0**% (**19/19**)
Co-Author `w:ins`/`w:del` & Tracked Nested Table	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19/19**)
Threaded Comments (`w15:paraIdParent`) & Ranges	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19/19**)
Overall Edited-Block Construct Survival (114)	0.0% (0/114)	16.7% (19/114)	**100.0**% (**114/114**)
Document-Wide `w14:paraId` Uniqueness ($N = 19$)	0.0% (0/19)	**100.0**% (19/19)[†]	**100.0**% (**19/19**)

[†]The XML `<w:p>` script preserves existing `w14:paraId` attributes when overwriting paragraphs in-place, whereas Heurion ADI additionally maintains 100% `w14:paraId` uniqueness when splitting paragraphs into paired `<w:del>`/`<w:ins>` Word tracked changes under `mode: "suggest"`.

Table 2: Full-manuscript operation latency across $N = 19$ clinical articles (mean 7,903 words, 143 AST blocks).

Full-Manuscript Engine Operation	**Mean (ms)**	**Median / P50 (ms)**	**P95 (ms)**
`.docx` Import, Comment Binding & CRDT Hydration	37.40	29.00	97.64
Side-Effect-Free Pre-Flight Probe (`doc_validate`)	**3.86**	**2.90**	**10.74**
Guarded Multi-Block CRDT Edit Batch (8 surgical ops)	24.26	17.42	67.84
In-Memory Selective AI Turn Undo (`revertTurn`)	**6.86**	**5.28**	**20.85**
Token-Patched `.docx` Export (`node_src` + `docx-patch`)	20.02	15.53	49.33

loses all publisher constructs (0/114). A careful `<w:pPr>`-preserving XML paragraph script retains 100.0% byte identity on the 2,553 untouched blocks and preserves outer `<w:sdt>` wrappers (19/19, 16.7% overall), yet destroys **100% (0/95) of intra-paragraph XML constructs** whenever a sentence inside those paragraphs is revised. By contrast, Heurion ADI preserves 100.0% byte identity across all 2,553 untouched blocks and 100.0% (114/114) of inline fields, footnotes, formulas, co-author tracked changes, and threaded comment ranges inside modified blocks.

Table 2 reports end-to-end latencies on full ∼7,900-word manuscripts: pre-flight validation completes in a median of 2.90 ms, selective CRDT turn undo in 5.28 ms, and full token-patched `.docx` export in 15.53 ms.

5.2 Experiment 2: Concurrent Co-Editing Contracts and Selective Turn Undo

Next, we evaluate three concurrency and undo architectures across 8 co-authoring scenarios on all 19 full-length manuscripts (152 full-document trials, plus span-safe coordinate tests on all 18 manuscripts containing clinical tables): (1) **Disk Overwrite** (unversioned file rewrite + snapshot restore), (2) **Naive Doc-Rev Lock** (standard document-level `base_rev == doc_rev` optimistic locking + linear snapshot undo), and (3) **Heurion ADI**.

As shown in Table 3, document-level `ETag` locking (**Naive Doc-Rev**) catches immediate stale-revision writes (S1: 100.0%), but fails completely (0.0%) when an agent acquires a fresh document revision on another block before overwriting an unread clinician edit (S2), when an agent summarizes away a clinician-corrected HR after reading (S3), when human and agent concurrently edit distinct clauses of the same paragraph (S4: rejected by coarse document locks),

Table 3: Human-AI co-editing contract and selective undo accuracy across $N = 19$ full-length clinical manuscripts.

Co-Authoring Scenario ($N = 19$ manuscripts each)	**Disk File Overwrite**	**Naive Doc-Rev (ETag + Linear)**	**Heurion ADI (Ours)**
S1: Stale `base_rev` Whole-Block Overwrite Blocked	0.0% (0/19)	**100.0**% (19/19)	**100.0**% (**19**/**19**)
S2: Unread Block Protected Under *Fresh* `base_rev` (`noteSeen`)	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
S3: Post-Read Clinician-Corrected Statistic (HR) Preserved	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
S4: Same-Paragraph Distinct-Clause Stale-Rev Commuting	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
S5: Open Comment Guard + Cross-Para Footnote/Comment `merge`	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
S6: Bare DOI, Native `[1]` Hijack & Fake `[@w:w99]` Blocked (57)	0.0% (0/57)	0.0% (0/57)	**100.0**% (**57**/**57**)
S7: Tier-1 Out-of-Order Intra-Paragraph `revertTurn` (User Safe)	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
S8: Tier-2 Cold-Restart SQLite `revertByNodes` (User Priority)	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
2D `TableGrid` Merge-Origin Write / Covered-Span Block ($N = 18$)	N/A	N/A	**100.0**% (**18**/**18**)

and when selectively reverting an earlier AI turn after intra-paragraph human edits (S7–S8). Heurion ADI resolves 100.0% of cases across S1–S8 and enforces 100.0% (18/18) span-safe cell addressing on multi-column clinical tables.

5.3 Experiment 3: Live Multi-Turn LLM Co-Editing and Within-Turn Self-Repair

Deterministic traces validate engine invariants, but leave open a practical HCI/systems question: when a real LLM drives multi-turn tool calls, can it interpret ADI's structured guard rejections (`code`, `hint`, `current`) and converge within the turn without corrupting `.docx` archives? We evaluate an unmodified cloud reasoning model (`deepseek-v4-flash`, Anthropic Messages tool-use protocol, $T = 0$, up to 5 steps/turn) across all $N = 19$ full-length PMC clinical manuscripts on three multi-step co-authoring workloads (57 episodes per arm; 171 live multi-turn episodes total):

- **W1 (Publisher-Rich Paragraph Revision):** Revise clinical wording and follow-up duration inside a paragraph containing an EndNote `ADDIN EN.CITE` complex field, a `<w:footnoteReference>`, and an inline `<m:oMath>` formula ($\alpha = 0.05$).
- **W2 (Mid-Session Concurrent Clinician Conflict):** In a realistic 3-turn session, the agent reads $\{p_1, p_2\}$ at r_0 (HR 1.48 in p_1), a clinician concurrently updates p_1 to HR 1.12 (r_1), and the agent updates p_2 in Turn 2 (receiving fresh r_2). In Turn 3, the user asks the agent to tighten p_1 via `replace_block` to report a 24-month window; success requires both applying the 24-month revision and preserving the clinician's concurrent HR 1.12 edit.
- **W3 (Merged Clinical Table Header + Anchored Comment):** Rename a merged header cell (`colspan`/`rowspan`) in each manuscript's clinical table and rewrite a prose sentence anchored by an open reviewer comment while maintaining strict `DOMParser` XML validity and balanced comment range markers on export.

We compare three live-LLM architectures under symmetric instructions: (1) **Direct OOXML LLM** (the model directly edits the target `<w:p>` and untruncated `<w:tbl>` XML elements with explicit instructions to preserve all ECMA-376 fields, IDs, comments, and merge properties), (2) **Naive Doc-Rev MCP** (multi-turn `doc_read`/`doc_edit` with standard `base_rev == doc_rev` locking and Markdown re-serialization export), and (3) **Heurion ADI MCP** (`doc_read` with `noteSeen`, `comments_list`, `doc_validate`, contract-guarded `doc_edit`, and `docx-patch` export).

Table 4 reveals three empirical findings. First, **Direct OOXML LLM** achieves 100.0% (19/19) on isolated single-paragraph edits (W1), confirming that modern reasoning LLMs can preserve inline XML fields when context is small and static; however, it drops to 0.0% under

Table 4: Live multi-turn LLM co-editing evaluation (`deepseek-v4-flash`, $T = 0$) across $N = 19$ full-length PMC manuscripts (3 workloads $\times 19 = 57$ live episodes per arm; 171 episodes total).

Live Agent Metric / Workload	**Direct OOXML LLM Rewrite**	**Naive Doc-Rev Block MCP**	**Heurion ADI Contract MCP**
W1: Publisher-Rich Paragraph Edit ($N = 19$)	**100.0**% (19/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
W2: Mid-Session Concurrent Clinician HR ($N = 19$)	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
W3: Merged Clinical Table + Open Comment ($N = 19$)	0.0% (0/19)	0.0% (0/19)	**100.0**% (**19**/**19**)
Overall End-to-End Task & Fidelity (57)	33.3% (19/57)	0.0% (0/57)	**100.0**% (**57**/**57**)
Strict Exported `.docx` XML Validity (57)	94.7% (54/57)	**100.0**% (57/57)	**100.0**% (**57**/**57**)
Pre-Commit Conflict Self-Repair on W2	N/A	0.0% (0/0 caught)	**100.0**% (**14**/**14**)[‡]
Mean Tool Steps / Episode	1.00	2.82	3.79
Mean Input / Output Tokens per Episode	3,401 / 3,446	1,741 / 546	2,279 / **798**
Mean Episode Latency (All / W3 Table)	11.17 s / 28.98 s	5.39 s / 6.43 s	7.53 s / **9.13** s

[‡]On W2, the agent proactively re-read $\{p_1\}$ before editing in 5/19 episodes; in the remaining 14/19 (73.7%) episodes it attempted `replace_block` under fresh `base_rev = 2`, hit `noteSeen`'s `conflict_user_edited` guard, and self-repaired in 14/14 (100.0%) cases (14/15 = 93.3% across all episodes recording any guard rejection, including one post-commit validation probe on W3).

concurrent clinician edits (W2) and 0.0% on full clinical tables (W3), where regenerating multi-row `<w:tbl>` XML consumes 3,446 mean output tokens (4.3× higher than Heurion ADI), incurs 28.98 s mean latency on W3 (3.2× slower than Heurion ADI's 9.13 s), and truncates mid-tag at the 16,384-token output ceiling on large tables (84.2% W3 XML validity). Second, **Naive Doc-Rev MCP** executes fast block edits (2.05 mean steps on W2) but achieves 0.0% (0/57) end-to-end fidelity: on W2, holding a fresh `doc_rev` from Turn 2 causes the agent to overwrite the clinician's unread HR 1.12 correction without warning in 19/19 episodes, while Markdown export strips inline fields and comment ranges on W1 and W3. Third, **Heurion ADI MCP** achieves 100.0% (57/57) task and OOXML fidelity: whenever the live agent attempts an unread rewrite on W2 (14/19 episodes), `noteSeen` intercepts the write and returns `extra.current`, enabling 100.0% (14/14) autonomous within-turn self-repair.

6 Related Work and Conclusion

Related Work. SWE-agent [1] and OpenHands [2] established Agent-Computer Interfaces for plain-text source code, while STORM [11] and PaperQA2 [12] automate read-only literature synthesis and CoAuthor [5] studies human-AI text generation in flat editors. On the collaborative systems side, Yjs [9] and Peritext [10] formalize rich-text CRDT convergence for human co-editors, but do not address asymmetric AI batch mutations, multi-turn agent read horizons, or lossless round-tripping with publisher OOXML [6] fields and tracked revisions. Heurion ADI bridges these two lines of work, turning collaborative CRDTs and token-patched OOXML into a contract-guarded interface for scientific agents.

Conclusion. Replacing free-form file scripting with a dual-surface Yjs CRDT core, per-block horizon and clinician-delta contracts, two-tier selective turn undo, and intra-paragraph token-patched OOXML export eliminates structural formatting loss and silent human-edit overwrites in collaborative clinical authoring, providing a clean systems foundation for higher-level scientific verification and review workflows.

References

[1] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated

software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[2] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[3] Kenneth F. Schulz, Douglas G. Altman, and David Moher. CONSORT 2010 statement: updated guidelines for reporting parallel group randomised trials. *BMJ*, 340:c332, 2010.

[4] Jan P. Vandenbroucke, Erik von Elm, Douglas G. Altman, Peter C. Gøtzsche, Cynthia D. Mulrow, Stuart J. Pocock, Charles Poole, James J. Schlesselman, and Matthias Egger. Strengthening the Reporting of Observational Studies in Epidemiology (STROBE): explanation and elaboration. *PLoS Medicine*, 4(10):e297, 2007.

[5] Mina Lee, Percy Liang, and Qian Yang. CoAuthor: Designing a human-AI collaborative writing dataset for exploring language model capabilities. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI)*, pages 1–19, 2022.

[6] Ecma International. Standard ECMA-376: Office Open XML File Formats (5th Edition). Ecma International, Geneva, 2021.

[7] William H. Walters and Esther Isabelle Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13(1):14045, 2023.

[8] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems (SSS)*, pages 386–400. Springer, 2011.

[9] Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. Near real-time peer-to-peer shared editing on extensible data types. In *Proceedings of the 19th International Conference on Supporting Group Work (GROUP)*, pages 39–49. ACM, 2016.

[10] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter Van Hardenberg. Peritext: A CRDT for rich-text collaboration. *Proceedings of the ACM on Human-Computer Interaction*, 6(CSCW2):1–36, 2022.

[11] Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[12] Michael D. Skarlinski, Sam Cox, Jon M. Laurent, James D. Braza, Collin Hinks, Michael J. Hammerling, Manvitha Ponnapati, Samuel G. Rodriques, and Andrew D. White. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv preprint arXiv:2409.13740*, 2024.